Eye-Tracking as a Game Input: Porting Reflexio to the Web and Evaluating Gaze-Based Control

Mengxiang Jiang
CSCE 624: Sketch Recognition Spring 2026

1 Introduction and Motivation

Alternative human-computer input modalities have long fascinated researchers and hobbyists alike. Moving beyond the conventional keyboard-and-mouse or gamepad paradigm opens the door to more accessible, expressive, or simply novel ways to interact with software. My interest in non-traditional game input began during my undergraduate studies, where I built a system for playing Pong using electroencephalography (EEG) signals recorded from a consumer headset [1]. That project demonstrated that exotic input channels can be made functional, but also highlighted the significant gap between "technically works" and "pleasant to use."

This project revisits that question in a different context: gaze-based control for a puzzle platformer. The game chosen is *Reflexio*, a tile-based puzzle platformer I originally wrote as an undergraduate course project in C# using the XNA/FNA framework [2]. Reflexio's signature mechanic is *reflection*: the player can mirror the entire level, or a subset of rows, columns, or diagonal bands, along a selectable line. Because choosing *which* reflection line to activate is a discrete, deliberate selection task rather than a continuous pointing task, it is a plausible candidate for gaze control; the player looks at the indicator for the desired axis and then uses a separate button press to trigger the reflection.

The goals of this project are therefore threefold: (1) port Reflexio to a web-accessible platform so that participants can play without installing anything; (2) integrate a webcam-based eye-tracking input channel for the reflection-line selection sub-task; and (3) collect qualitative feedback on whether the gaze control improves, impairs, or simply does not affect the playing experience.

2 Related Background

2.1 Eye-Tracking in HCI and Games

Eye-tracking has been studied as a computer input modality since at least the 1990s [3]. The core challenge in using gaze as a *command* channel is the **Midas touch problem** [4]: because users look at many things they do not intend to select, a purely gaze-triggered system fires spuriously. The canonical mitigation is to require a secondary confirmation gesture (a button press, dwell timer, or blink) before any action is executed. This project adopts the button-press approach: the player uses gaze to navigate among reflection-line indicators, and a separate key or button to execute the reflection.

2.2 Saccades and Smooth Pursuit

Eye movements fall into two primary categories relevant here. **Saccades** are rapid, ballistic jumps between fixation points, typically lasting 20–200 ms. **Fixations** are the relatively stable periods between saccades, during which the visual system processes information. Consumer webcam-based trackers such as WebGazer.js operate at video frame rates (typically 30 fps) and return the estimated screen coordinate of the current fixation. The relatively coarse temporal resolution means that saccade onset and offset cannot be detected precisely; instead, a change in fixation region must be inferred from a sustained shift in the gaze estimate.

2.3 Fitts's Law

Fitts's law [5] predicts the time required to move a pointer to a target as a function of the distance to the target and its width: $T = a + b \log_2(1 + D/W)$. For gaze-based pointing, the effective "width" of a target is limited by tracker accuracy rather than target size, which severely penalises small or closely spaced targets. The reflection-line indicators in Reflexio are relatively small, making gaze pointing challenging even in principle.

2.4 Kalman Filtering for Gaze Smoothing

Raw webcam-based gaze estimates are noisy. A scalar Kalman filter [6] is a standard tool for smoothing a one-dimensional noisy measurement sequence under a linear-Gaussian model. Applying independent filters to the x and y gaze coordinates reduces high-frequency jitter while introducing a small lag proportional to the process-noise-to-measurement-noise ratio. This project applies a 1-D Kalman filter to each axis of the WebGazer output.

2.5 WebGazer.js

WebGazer.js [7] is an open-source JavaScript library developed at Brown University that performs real-time gaze estimation using only a standard webcam accessed through the browser's `getUserMedia` API. It uses facial landmark detection to locate the eyes and trains a regression model (ridge regression by default) mapping eye-region image features to screen coordinates. Training data is collected implicitly as the user interacts with the page (click and cursor positions serve as ground-truth labels). No specialised hardware is required, making it well suited for a web-deployed experiment.

3 Method and Approach

The project proceeded in two phases.

Phase 1 – Web port. The original Reflexio codebase targets the XNA/FNA desktop runtime and cannot run in a browser. The first task was therefore to produce a faithful web port. After evaluating several JavaScript game frameworks, Phaser 3 [8] was selected for its built-in Matter.js physics engine, rich sprite and animation support, and active community. The port preserves all 53 published levels, the core reflection mechanic, player movement and physics, key/door/switch puzzles, and the main menu and level selection flow. Mobile touch controls were added later in response to user feedback (Section 5).

Phase 2 – Gaze input for reflection selection. The reflection-line selection sub-task was identified as the most suitable target for gaze input because it is a discrete choice among a small number of options (horizontal, vertical, or diagonal axes) and the Midas touch problem is naturally mitigated: looking at an indicator changes the *highlighted* line but does not execute the reflection; a separate button press is required. This matches the intent-plus-confirmation pattern recommended in the gaze-interaction literature.

Gaze is used to cycle through reflection line categories via detected saccades. A large horizontal saccade (displacement exceeding a calibrated threshold) cycles the horizontal reflection lines; a large vertical saccade cycles the vertical lines. Diagonal lines are selected by keyboard or gamepad as a fallback. An auto-cycling feature was added so that holding a saccade direction repeatedly advances through the available lines without requiring repeated saccades.

4 Implementation Details

4.1 Game Port

The port was implemented in JavaScript using Phaser 3 and bundled with Vite. The original C# codebase used reflection (in the language sense) to instantiate game objects from XML level files; this was replicated in JavaScript via a class registry that maps XML tag names to constructors. All 64 level XML files from the original ship unchanged. Physics bodies are provided by Matter.js: walls and most objects use static rectangle bodies, the player uses a 20-vertex polygon approximating an ellipse (matching the original's `PolygonTools.CreateEllipse`), and grabbable blocks use dynamic bodies connected to the player via a distance constraint.

A significant portion of the implementation work involved learning JavaScript and the Phaser API. Claude (Anthropic's AI assistant) was used extensively as a coding aid throughout the port, providing API guidance, debugging support, and code generation for boilerplate-heavy sections.

4.2 Touch and Mobile Controls

Following a request from a friend who discovered the game on his phone, a DOM overlay providing virtual controls was added. The overlay uses Pointer Events (which unify mouse and touch input) and positions all elements with CSS `calc()` expressions in viewport units so that the layout adapts to any screen size without JavaScript measurement. Two D-pads handle movement and reflection-line navigation; A and B buttons handle jump and reflect. The browser's Fullscreen API and Screen Orientation API are invoked on the first user gesture to maximise the playable area and lock landscape orientation on mobile devices.

4.3 Eye-Tracking Integration

WebGazer.js is loaded on demand from a CDN when the user enables the eye tracker. After `webgazer.begin()` acquires camera permission and starts the regression model, raw gaze coordinates are post-processed by two independent 1-D Kalman filters (one per axis). The filter parameters were tuned empirically to balance lag against noise reduction.

The smoothed gaze point is compared each frame to a running baseline (an exponential moving average of recent gaze positions). When the displacement from the baseline exceeds a threshold—100 pixels horizontally or 60 pixels vertically—a saccade is registered and the corresponding reflection-line category is advanced. A 120-frame cooldown prevents multiple triggers from a single saccade. Outlier rejection discards any single-frame jump exceeding 300 pixels, which typically corresponds to tracker loss or a blink rather than a genuine saccade.

Blink detection is implemented by counting consecutive null callbacks from WebGazer (indicating the eye region is occluded). A run of 2–18 null samples is classified as a blink. Blinks were initially considered as a potential command channel but were ultimately not used due to reliability concerns.

The webcam preview and WebGazer's built-in prediction-point overlay are displayed in a fixed corner of the screen at reduced opacity so the player can monitor tracker state without it dominating the view. Eye-tracking buttons (toggle and calibrate) are suppressed on mobile browsers, where webcam-based gaze tracking is impractical, using a user-agent check.

4.4 Kalman Filter

The 1-D Kalman filter implementation maintains state $(\hat{x}, P)$ representing the estimated gaze coordinate and its error covariance. Each frame the prediction step advances the state with process noise $Q = 0.1$ and the update step incorporates the WebGazer measurement with measurement noise $R = 30$. These values were chosen so that the filter follows genuine fixation shifts within roughly five frames while attenuating single-frame noise spikes.

5 Results and Evaluation

5.1 Tracking Accuracy

Initial testing of the raw WebGazer output revealed gaze estimation errors exceeding 100 pixels on a 1920×1080 display. The reflection-line indicators in Reflexio are small (on the order of 30–40 pixels), making accurate pixel-level selection infeasible. Applying the Kalman filter reduced the effective jitter to approximately 50 pixels, which is an improvement but still insufficient for reliable indicator selection by fixation alone.

5.2 Saccade-Based Selection

Switching from fixation-based to saccade-based selection significantly reduced the accuracy demands: the user need only produce a large-magnitude eye movement in the correct cardinal direction, rather than fixating a specific small target. In informal testing this was more reliable than fixation-based selection. However, the discretisation (horizontal saccade → cycle horizontal lines; vertical saccade → cycle vertical lines) means the user must repeat the same saccade multiple times to step through several lines of the same orientation, which is fatiguing and inconsistent. The auto-cycling feature (advancing automatically every two seconds after a saccade is registered) partially mitigates this but introduces unwanted advances when the player is not actively selecting.

5.3 User Feedback

The author and three friends played Reflexio with eye-tracking enabled. The unanimous reaction was that the gaze control made the game noticeably harder and less enjoyable to play. Specific complaints included:

- Calibration drift: tracking quality degraded over a session, requiring frequent manual recalibration.
- Cognitive load: monitoring gaze direction in addition to the in-game state was mentally taxing.
- Misdirected saccades: glancing at score displays or the webcam preview triggered unintended line changes.
- Inconsistency: the same intentional saccade did not always produce the same result.

One participant who tested the game on a mobile phone noted the absence of touch controls. This feedback directly motivated the implementation of the virtual D-pad overlay described in Section 4.2, which was well received.

6 Discussion and Limitations

The central finding is that eye-tracking input, at least as implemented here with consumer-grade webcam-based tracking, is not a viable replacement for keyboard or gamepad input in a fast-paced puzzle platformer. The accuracy limitations of WebGazer.js on typical hardware are a fundamental constraint that no amount of software-side smoothing can fully overcome; the technique requires purpose-built hardware (infrared-illuminated eye trackers with sub-pixel accuracy) to reach the precision needed for reliably selecting small on-screen targets.

The choice of Reflexio's reflection mechanic as the target sub-task was deliberate and reasonable: it is a discrete choice task, it naturally avoids the Midas touch problem, and it does not require continuous high-bandwidth gaze control. Nevertheless, even this limited use of gaze proved frustrating in practice, primarily due to calibration drift and the difficulty of producing consistent, repeatable saccades.

Several limitations constrain the generalisability of these results. The evaluation was informal, with three participants, no control condition, and no quantitative performance metrics such as level completion time or error rate. A more rigorous study would include a within-subjects comparison of keyboard-only versus keyboard-plus-gaze conditions, objective performance measures, and a larger and more diverse participant pool. The port itself, while faithful in mechanics, has not been systematically validated against the original for physics fidelity across all 53 levels.

7 Conclusion

This project produced a complete web port of the puzzle platformer Reflexio, playable at https://kumj2028.github.io/reflexio-phaser/ with full mobile touch control support, and integrated webcam-based eye tracking for the reflection-line selection sub-task. The eye-tracking integration demonstrated that novelty of input modality does not imply usability: participants consistently found the gaze control more burdensome than the standard keyboard alternative. The technical limitations of consumer-grade webcam tracking—high noise, calibration drift, and sensitivity to ambient lighting—impose accuracy constraints that are difficult to overcome in software. The project reinforces a well-established principle in HCI: a new control scheme must improve the user experience in a measurable way; if the existing scheme works well, replacing it with a novel but inferior alternative degrades rather than enhances the game. The most practically useful outcome of the project was not the eye-tracking integration itself but the mobile touch control overlay, which extended the game's accessibility to smartphone users and received positive feedback.

Source code is available at https://github.com/kumj2028/reflexio-phaser.

References

[1] C. Moyes and M. Jiang, "EEG-Controlled Pong," ECE 4760 Final Project, Cornell University, Spring 2012. https://people.ece.cornell.edu/land/courses/ece4760/FinalProjects/s2012/cwm55/cwm55_mj294/

[2] M. Jiang, "Reflexio," CS 3152 course project, Cornell University. https://kumj2028.github.io/CS3152/index.html

[3] R. J. K. Jacob, "The use of eye movements in human-computer interaction techniques: What you look at is what you get," *ACM Transactions on Information Systems*, vol. 9, no. 2, pp. 152–169, 1991.

[4] R. J. K. Jacob, "What you look at is what you get: eye movement-based interaction techniques," in *Proc. ACM CHI*, 1990, pp. 11–18.

[5] P. M. Fitts, "The information capacity of the human motor system in controlling the amplitude of movement," *Journal of Experimental Psychology*, vol. 47, no. 6, pp. 381–391, 1954.

[6] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.

[7] A. Papoutsaki, P. Sangkloy, J. Laskey, N. Daskalova, J. Huang, and J. Hays, "WebGazer: Scalable webcam eye tracking using user interactions," in *Proc. IJCAI*, 2016. https://webgazer.cs.brown.edu/

[8] Photon Storm Ltd., "Phaser 3," 2024. https://phaser.io/